TECHNICAL PROJECT REPORT

Real-Time Platform Occupancy Detection for Railway Stations

Using Computer Vision and YOLOv8 Deep Learning

Domain: Intelligent Transportation Systems | AI & Computer Vision
Classification: Academic / Industry Reference
Version: 1.0 | Year: 2025

Table of Contents

1. Abstract 3
2. Introduction 3
3. Problem Statement 4
4. Objectives 4
5. Literature Review 5
6. Proposed System 6
7. System Architecture 7
8. Methodology 8
9. Technology Stack 9
10. Implementation Plan 11
11. Expected Output 12
12. Advantages 13
13. Limitations 13
14. Future Enhancements 14
15. Applications 15
16. Conclusion 15

Appendix A. Workflow Diagram Explanation 16

Appendix B. Pseudo-Code: Crowd Counting Logic 17

1. Abstract

Railway platforms are dynamic environments that regularly experience extreme crowd fluctuations, particularly during peak commute hours, festive seasons, and unscheduled service disruptions. The inability to monitor occupancy levels in real time poses significant risks to passenger safety and operational efficiency. This report presents the design and implementation of a Real-Time Platform Occupancy Detection System that leverages state-of-the-art Computer Vision techniques and deep learning to continuously analyse live CCTV video feeds, detect individual passengers, count crowd density, and classify occupancy into actionable categories.

The proposed system is built upon the YOLOv8 object detection architecture developed by Ultralytics, which delivers exceptional inference speed and detection accuracy even in congested environments. Video frames are captured and processed through a modular pipeline comprising frame extraction, human detection, person counting, density classification, and live dashboard visualisation. The resulting application enables railway operations staff to receive real-time occupancy alerts, facilitating timely crowd management interventions such as platform closures, announcement broadcasts, or additional service deployment.

The core technologies employed include Python as the primary programming language, OpenCV for video frame processing, YOLOv8 for object detection, FastAPI or Flask as the backend service layer, and Streamlit for the interactive monitoring dashboard. The system is designed to be lightweight, scalable, and deployable on standard edge computing hardware without requiring dedicated GPU infrastructure in constrained settings. This document presents the full system architecture, methodology, technology rationale, implementation plan, and anticipated outcomes of the project.

Keywords: *Computer Vision, YOLOv8, Crowd Detection, Railway Safety, Real-Time Monitoring, Object Detection, Density Classification, Intelligent Transportation Systems*

2. Introduction

2.1 Background

Railway networks form the backbone of urban mobility across the world, transporting billions of passengers annually across dense metropolitan corridors and intercity routes. In countries such as India, Japan, China, and the United Kingdom, rail infrastructure operates at or near maximum capacity for extended portions of the day. Platform areas, in particular, serve as transitional zones where passenger flow is inherently unpredictable: surges occur when delayed trains arrive, when special events conclude, or when weather conditions drive commuters indoors.

The consequences of unmanaged crowd congestion on railway platforms range from passenger discomfort and delayed boarding to catastrophic safety incidents including crowd crushes, falls onto tracks, and physical altercations. Historical tragedies, such as the 2010 Duisburg Love Parade stampede and recurring platform overcrowding incidents at major urban stations, underscore the critical need for automated, real-time crowd surveillance. Traditional approaches to crowd management — manual observation by station staff or periodic headcounts — are inherently reactive, subjective, and operationally expensive.

2.2 Importance of Real-Time Monitoring

The advent of deep learning-based Computer Vision has introduced transformative possibilities for automated crowd analysis. Modern Convolutional Neural Network (CNN) architectures can process high-resolution video at real-time frame rates, enabling continuous, objective surveillance without human intervention. When integrated with an alerting and dashboard system, such technology allows station managers to respond proactively rather than reactively to emerging crowd situations.

Real-time monitoring systems provide several distinct advantages over manual surveillance: they operate without fatigue, can simultaneously monitor multiple camera feeds, generate quantitative data for capacity planning, and can be configured to trigger automated alerts when occupancy thresholds are breached. These capabilities align directly with the objectives of smart city initiatives and intelligent transportation systems (ITS) frameworks being adopted by railway operators worldwide.

2.3 Motivation

The motivation for this project arises from the practical gap between the widespread availability of CCTV infrastructure in railway stations and the limited analytical intelligence applied to those video feeds. In most stations, CCTV cameras serve a

primarily retrospective security function, with footage reviewed only after an incident has occurred. This project aims to bridge that gap by developing a system that transforms passive surveillance infrastructure into an active, intelligent occupancy monitoring asset. The project further aims to demonstrate that such a system can be constructed using widely accessible, open-source tools and pre-trained models, making it accessible to transit authorities with varying levels of technical resources.

3. Problem Statement

Railway platforms are among the most densely occupied public spaces in urban environments, yet the tools available to station operators for managing those crowds remain largely manual and reactive. The fundamental problem addressed by this project can be articulated as follows: there is no automated, real-time mechanism currently deployed at most railway stations to continuously monitor the number of passengers on a platform, classify the level of crowding, and deliver actionable alerts to operations staff before a safety threshold is breached.

Manual monitoring presents a series of inherent challenges. First, human observers stationed on platforms are limited in their field of view and cannot simultaneously monitor multiple platform zones. Second, subjective visual estimation of crowd density by staff members introduces significant variability — what one operator judges to be dangerously crowded may not be assessed similarly by another. Third, fatigue substantially diminishes the reliability of manual monitoring during extended shifts. Fourth, during incidents where rapid crowd surges occur — such as when a long-delayed train arrives — staff reaction time is often insufficient to implement preventive measures before unsafe conditions emerge.

Existing CCTV infrastructure, while extensive, is predominantly used for post-incident review rather than active monitoring. Control room operators responsible for monitoring multiple feeds simultaneously face cognitive overload that reduces the practical effectiveness of human-operated surveillance. Furthermore, no standardised, quantitative occupancy metric is generated from these feeds, preventing data-driven capacity planning and historical analysis.

The challenge is therefore threefold: to extract meaningful, quantitative occupancy data from live video in real time; to classify that data into actionable crowd density categories; and to present the results through an interface that is intuitive and responsive enough to support immediate operational decision-making. Additionally, the system must contend with the practical challenges inherent to video-based crowd analysis in railway environments, including partial occlusion of passengers by pillars, overhead structures, or other passengers; variable lighting conditions between day and night operations; and the presence of non-human objects such as luggage and equipment that may confuse detection algorithms.

4. Objectives

4.1 Primary Objective

The primary objective of this project is to design, develop, and deploy a real-time platform occupancy detection system that uses Computer Vision and deep learning to automatically detect, count, and classify the density of passengers on railway station platforms from live or recorded CCTV video feeds.

4.2 Sub-Objectives

The following sub-objectives collectively define the scope and success criteria of the project:

- To implement a robust person detection pipeline using the pre-trained YOLOv8 model, capable of identifying individual passengers in complex, crowded environments with high precision and recall.
- To develop a real-time person counting module that accurately enumerates detected individuals in each processed video frame, accounting for partial occlusions and varying camera perspectives.
- To design and apply a density classification algorithm that categorises the current occupancy level of a platform segment as Low, Medium, or High based on configurable passenger count thresholds.
- To build a backend API service that receives processed occupancy data from the computer vision pipeline and exposes it to connected dashboard clients.
- To develop a live, web-based monitoring dashboard that displays real-time occupancy metrics, crowd density classifications, and visual indicators for operations staff.
- To implement a threshold-based alert mechanism that generates notifications when occupancy levels exceed predefined safety limits.
- To ensure the system architecture is modular and scalable, permitting future extension to multi-camera deployments, historical data logging, and predictive analytics.
- To validate the system's detection accuracy and latency performance against representative railway platform video footage.

These objectives collectively position the system as a complete, production-oriented solution rather than a proof-of-concept prototype, ensuring its applicability to real-world railway operations upon deployment.

5. Literature Review

5.1 Overview of Crowd Analysis Research

The academic field of automated crowd analysis has grown substantially over the past two decades, driven by advances in machine learning, GPU computing, and the proliferation of surveillance camera infrastructure. Early approaches to crowd density estimation relied on traditional image processing techniques, such as optical flow analysis, background subtraction, and texture-based feature extraction. While computationally efficient, these methods demonstrated limited robustness under the variable conditions typical of real-world deployments, particularly when dealing with high-density crowds where individual detection becomes infeasible.

5.2 Deep Learning-Based Approaches

The introduction of deep Convolutional Neural Networks (CNNs) fundamentally transformed crowd analysis research. Zhang et al. (2016) proposed the Multi-column CNN (MCNN) architecture for crowd counting, which employs multiple parallel convolutional columns with different receptive field sizes to accommodate the scale variability of pedestrians within a single image. This approach introduced the concept of density map estimation, in which a neural network predicts a spatial density map rather than individual bounding boxes, thereby addressing the occlusion challenge in high-density scenarios.

Subsequent work by Cao et al. (2018) introduced the Scale Aggregation Network (SANet), which further improved counting accuracy by aggregating multi-scale features. Li et al. (2018) proposed CSRNet, which utilises dilated convolutional layers to expand the receptive field without increasing the number of parameters, achieving state-of-the-art performance on benchmark datasets such as ShanghaiTech and UCF-CC-50.

For real-time person detection, the YOLO (You Only Look Once) family of architectures has emerged as the dominant approach due to its ability to perform detection in a single forward pass, enabling inference speeds sufficient for video analysis. Redmon et al. (2016) introduced the original YOLO architecture, which framed object detection as a regression problem over a spatial grid. Subsequent versions — YOLOv3, YOLOv5, and the current YOLOv8 released by Ultralytics in 2023 — have progressively improved accuracy, speed, and ease of deployment.

5.3 Existing Systems and Products

Several commercial and research systems have addressed railway crowd monitoring. The British Transport Police deployed video analytics systems at major London terminals following the 2005 bombings, primarily for security rather than capacity management purposes. Tokyo Metro employs proprietary AI-based crowd monitoring at key stations, integrated with dynamic passenger information systems. Siemens Mobility has developed AI-based platform monitoring solutions integrated into its Railigent platform, though these are predominantly targeted at large infrastructure operators with significant capital budgets.

Research prototypes have also been demonstrated in academic literature. Patel and Shah (2021) proposed a CCTV-based platform occupancy system using Faster R-CNN for detection and a custom counting module, achieving satisfactory accuracy but operating below real-time frame rates on standard CPU hardware. Singh et al. (2022) explored the use of YOLOv5 for railway station crowd monitoring, demonstrating improved inference speed with acceptable detection accuracy, though the system lacked an integrated dashboard and alert mechanism.

5.4 Limitations of Current Solutions

A consistent limitation across both commercial and research systems is their reliance on expensive, proprietary hardware and software stacks, which restricts accessibility to smaller transit authorities. Density map estimation methods, while effective at overall count accuracy in very high-density scenarios, do not localise individual passengers, limiting their utility for applications requiring per-person tracking or spatial distribution analysis. Real-time performance on edge hardware remains a challenge for heavier architectures such as Faster R-CNN and its variants. Furthermore, most existing systems operate as monolithic, closed solutions without the modular, API-first design philosophy required for integration into broader smart station management ecosystems.

The proposed system addresses these limitations by employing YOLOv8, which achieves real-time inference on standard hardware, adopting an open-source stack that minimises deployment cost, and implementing a modular API-dashboard architecture that supports integration and future extensibility.

6. Proposed System

6.1 System Overview

The proposed Real-Time Platform Occupancy Detection System is an end-to-end computer vision pipeline that ingests live CCTV video feeds, applies deep learning-based human detection, counts the number of passengers visible in each frame, classifies the occupancy level, and presents the results through a live web dashboard. The system is designed to be modular, enabling independent development, testing, and replacement of individual components without disrupting the overall pipeline.

At a high level, the system comprises three primary layers: a processing layer responsible for video ingestion and computer vision inference; a service layer that exposes processed data through a RESTful API; and a presentation layer that renders occupancy metrics and alerts through a browser-accessible dashboard.

6.2 Justification of Approach

The decision to employ YOLOv8 for person detection is grounded in its demonstrated superiority along the accuracy-speed trade-off curve relative to alternative architectures available at the time of development. YOLOv8 achieves mean average precision (mAP) scores competitive with substantially heavier two-stage detectors such as Faster R-CNN, while delivering inference speeds of 30 frames per second or higher on a standard NVIDIA GPU, and acceptable performance on modern CPU hardware. Its pre-training on the COCO dataset, which includes a person class with over 250,000 annotated instances, means that the model arrives with robust generalised person detection capability without requiring custom dataset collection and training — a significant practical advantage for organisations without dedicated machine learning resources.

The choice of density classification over density map estimation reflects the operational requirements of the target use case. Railway operations staff require a simple, interpretable occupancy signal rather than a continuous density distribution. A three-tier classification system — Low, Medium, High — maps directly onto the operational responses available to station managers (routine operation, increased staff attention, platform access restriction), making the system immediately actionable without requiring specialist interpretation.

The API-first architecture, with Streamlit serving as the dashboard frontend and FastAPI or Flask providing the backend service, ensures that occupancy data can be consumed by multiple clients simultaneously, including the web dashboard, mobile

applications, and third-party station management systems. This design decision future-proofs the system against evolving operational requirements and integration needs.

6.3 Why This System is Novel

While individual components of the proposed system — YOLOv8 detection, OpenCV video processing, Streamlit dashboarding — are themselves established technologies, their integration into a unified, production-ready platform occupancy monitoring system tailored to the specific operational context of railway stations constitutes the novel contribution of this project. The system's open-source foundation, modularity, real-time performance, and contextualised classification logic collectively differentiate it from both existing commercial solutions and prior academic prototypes.

7. System Architecture

7.1 End-to-End Pipeline Overview

The system architecture follows a linear processing pipeline, from raw video input through to dashboard display, with each stage performing a clearly defined transformation on the data. The pipeline comprises six primary modules: Video Ingestion, Frame Extraction, YOLOv8 Detection, Person Counting, Density Classification, and Dashboard Visualisation. A seventh, cross-cutting module — the API Service Layer — connects the processing pipeline to the presentation layer.

```
[ CCTV Feed / Video File ] → [ Frame Extractor ] → [ YOLOv8 Detector ] → [ Person Counter ] → [ Density Classifier ] → [ API Service ] → [ Streamlit Dashboard ]
```

7.2 Module Descriptions

7.2.1 Video Ingestion Module

The Video Ingestion Module interfaces with the video source, which may be a live RTSP stream from a network-attached CCTV camera, a local USB camera feed, or a pre-recorded video file for testing and validation purposes. OpenCV's VideoCapture class is used as the primary interface for all source types, providing a unified API that abstracts the differences between live streaming protocols and file-based inputs. The module initialises the video source and continuously reads frames, passing them to the Frame Extraction Module.

7.2.2 Frame Extraction Module

Processing every frame captured from a 25 or 30 fps video stream would impose unnecessary computational load without proportional benefit, since crowd density changes over seconds rather than fractions of a second. The Frame Extraction Module therefore implements configurable frame skipping, processing only every Nth frame — typically every second or third frame at 30 fps — to achieve a target processing rate of 10 to 15 effective frames per second. Extracted frames are resized to the input resolution expected by the YOLOv8 model (640×640 pixels by default) and buffered for detection.

7.2.3 YOLOv8 Detection Module

The YOLOv8 Detection Module loads a pre-trained YOLOv8 model — specifically the YOLOv8n (nano) or YOLOv8s (small) variant for CPU-constrained deployments, or YOLOv8m (medium) for GPU-enabled servers requiring higher accuracy. The model processes each extracted frame and returns a list of detected objects, each represented by a bounding box, a class label, and a confidence score. The Detection Module filters these results to retain only detections with the class label 'person' and a confidence score above a configurable threshold (default: 0.40) to suppress false positives.

7.2.4 Person Counting Module

The Person Counting Module receives the filtered list of person detections from the Detection Module and computes the total count for the current frame. In basic implementations, this is a direct count of the filtered detection list. In advanced configurations, the counting module may incorporate a region of interest (ROI) mask to restrict counting to a specific platform zone, or employ a simple centroid tracker to count unique individuals rather than instantaneous detections, thereby mitigating the effect of frame-to-frame detection variability. The computed count is timestamped and passed to the Density Classification Module.

7.2.5 Density Classification Module

The Density Classification Module maps the person count to one of three occupancy categories using configurable count thresholds. By default, the thresholds are set as follows: a count of fewer than 15 persons is classified as Low density; a count between 15 and 40 persons (inclusive) is classified as Medium density; and a count exceeding 40 persons is classified as High density. These thresholds are intended as starting values and should be calibrated to reflect the physical dimensions and normal capacity of the specific platform being monitored. The module outputs the classification label, the person count, and a timestamp.

7.2.6 API Service Layer

The API Service Layer is implemented using FastAPI (or Flask, as an alternative) and exposes the processed occupancy data through a set of RESTful endpoints. The primary endpoint, GET /occupancy, returns the most recent person count, density classification, and timestamp as a JSON response. An optional WebSocket endpoint enables the dashboard to receive real-time push updates without polling. The API layer also exposes a configuration endpoint that allows authorised clients to modify detection thresholds and classification boundaries at runtime without restarting the system.

7.2.7 Dashboard (Presentation Layer)

The Streamlit-based Dashboard provides a live, browser-accessible interface for operations staff. The dashboard displays the current person count, the density classification with colour-coded visual indicators (green for Low, amber for Medium, red for High), a time-series chart of occupancy over the preceding period, and any active alerts. The dashboard auto-refreshes at a configurable interval and is accessible from any networked device within the station's local area network, requiring no client-side installation.

8. Methodology

8.1 Overall Process

The methodology of the system operates as a continuous, iterative loop: frames are continuously read from the video source, processed through the detection and counting pipeline, classified, and the results are made available via the API. The loop repeats for each extracted frame until the video source is exhausted or the system is manually terminated.

8.2 YOLOv8 Detection Algorithm

YOLOv8 implements a single-stage, anchor-free object detection architecture. Given an input image resized to 640×640 pixels, the model applies a backbone network (CSPDarknet53 with C2f modules in YOLOv8) to extract hierarchical feature representations at multiple scales. A feature pyramid network (FPN) with path aggregation (PANet) fuses these multi-scale features to produce representations sensitive to both small and large objects. The detection head then predicts, for each cell in a spatial grid, the probability of an object being centred in that cell, the regression offsets for the bounding box dimensions, and class probabilities across all 80 COCO categories.

During inference, the raw predictions are post-processed using Non-Maximum Suppression (NMS), which eliminates redundant overlapping bounding boxes by retaining only the detection with the highest confidence score when multiple boxes share an intersection-over-union (IoU) ratio exceeding a defined threshold (default: 0.45). The resulting set of non-overlapping, high-confidence detections represents the model's final output for a given frame.

8.3 Counting and Classification Logic

Following detection, the system counts the number of bounding boxes assigned to the person class and applies the density classification thresholds. A smoothing function, implemented as an exponentially weighted moving average over the preceding five frames, is applied to the raw count before classification to reduce the visual jitter of rapidly changing counts in the dashboard. The smoothed count is classified against the threshold boundaries, and the result is emitted to the API service.

8.4 Performance Considerations

On an NVIDIA GPU (e.g., RTX 3060 or higher), YOLOv8n achieves inference latency of approximately 2–5 milliseconds per frame, comfortably supporting real-time processing at 30 fps. On a modern CPU (e.g., Intel Core i7 8th generation or equivalent), inference latency is approximately 50–100 milliseconds per frame, limiting effective

processing to 10–20 fps. Frame skipping mitigates this constraint, ensuring that the system operates within acceptable latency bounds on CPU-only hardware. For production deployments, edge computing devices such as the NVIDIA Jetson Nano or Raspberry Pi 5 with HAILO AI accelerator provide a cost-effective middle ground.

9. Technology Stack

The technology stack of the proposed system has been carefully selected to balance performance, accessibility, community support, and long-term maintainability. Each technology choice is justified below on the basis of its specific contribution to the system's requirements.

Technology	Role in System	Justification
Python 3.10+	Primary programming language	Ecosystem dominance in ML/CV; extensive library support
OpenCV 4.x	Video capture & frame processing	Industry standard; supports RTSP, USB, and file sources
Ultralytics YOLOv8	Object detection model	State-of-the-art real-time accuracy; simple API
FastAPI / Flask	Backend REST API	Lightweight; async support; easy deployment
Streamlit	Live monitoring dashboard	Rapid UI development; auto-refresh; Python-native
NumPy	Numerical computation	Core dependency for array operations on frames
Torch / PyTorch	YOLOv8 inference runtime	De-facto DL framework; CUDA support
Docker	Containerised deployment	Environment consistency; scalable orchestration

9.1 Python

Python serves as the primary programming language for the entire system. Its dominance in the machine learning and computer vision communities means that virtually all required libraries — from deep learning frameworks to video processing utilities — are available as mature, well-maintained Python packages. Python's interpreted nature facilitates rapid prototyping and iteration, while its extensive documentation and community support reduce the friction of debugging and extending the system.

9.2 OpenCV

OpenCV (Open Source Computer Vision Library) is the de-facto standard for video capture and image processing in Python applications. It provides a unified interface for reading video from diverse sources — including RTSP network streams, local USB cameras, and video files — through the VideoCapture class. OpenCV also provides essential preprocessing functions, including image resizing, colour space conversion

(BGR to RGB), and bounding box drawing, which are used throughout the processing pipeline. Its C++ backend ensures efficient memory management and processing speed even when called through the Python API.

9.3 Ultralytics YOLOv8

YOLOv8, released by Ultralytics in January 2023, represents the current generation of the YOLO (You Only Look Once) detection family. It provides a clean, task-oriented Python API that abstracts the complexity of model loading, inference, and result parsing into a few lines of code. The pre-trained COCO weights for the person class are directly applicable to the task of passenger detection without requiring any fine-tuning, significantly reducing the time and expertise required for deployment. YOLOv8 supports multiple model scales (n, s, m, l, x), allowing the system to be configured for the available hardware. Its support for ONNX and TensorRT export formats facilitates optimised deployment on edge hardware.

9.4 FastAPI / Flask

FastAPI is selected as the preferred backend framework due to its native support for asynchronous request handling, automatic OpenAPI documentation generation, and superior performance relative to synchronous alternatives such as Flask. Asynchronous handling is particularly important in this context because the API must serve dashboard clients while simultaneously receiving updates from the computer vision processing loop. Flask is offered as an alternative for environments where synchronous processing is preferred or where development team familiarity favours the simpler Flask architecture. Both frameworks are lightweight, requiring minimal configuration to deploy as standalone services.

9.5 Streamlit

Streamlit is selected for the dashboard frontend because it enables the construction of interactive, data-driven web applications entirely in Python, eliminating the need for separate JavaScript development. Its st.metric, st.line_chart, and st.empty widgets provide the visual components required for the occupancy dashboard, and its auto-refresh functionality (via st_autorefresh or custom polling) ensures that the displayed data remains current. Streamlit applications can be served on a local network or deployed to Streamlit Community Cloud for broader accessibility without configuration overhead.

9.6 Docker and Deployment Options

Docker containerisation is specified as the recommended deployment mechanism for production environments. Packaging the application in a Docker container ensures environment consistency across development, testing, and production deployments, and enables the use of orchestration tools such as Docker Compose or Kubernetes for

multi-container deployments (e.g., separate containers for the video processor and the API server). Alternative deployment options include direct installation on a Windows or Linux host machine for development and validation, and deployment on cloud platforms such as AWS EC2 or Google Cloud Compute Engine for centralised multi-station monitoring.

10. Implementation Plan

10.1 Development Phases

The implementation of the system is structured into five sequential development phases, each building upon the deliverables of the preceding phase. This phased approach enables incremental validation and reduces the risk of compound errors propagating through the pipeline.

Phase	Activities	Estimated Duration
Phase 1: Environment Setup	Install Python, OpenCV, YOLOv8, FastAPI, Streamlit; configure virtual environment; validate GPU/CPU inference	1 week
Phase 2: Core Detection Pipeline	Implement video ingestion, frame extraction, YOLOv8 inference, person filtering; validate on sample video	2 weeks
Phase 3: Counting & Classification	Develop counting module with ROI support; implement density classifier; validate accuracy on test footage	1 week
Phase 4: API & Dashboard	Build FastAPI backend; develop Streamlit dashboard with live metrics, charts, and alerts	2 weeks
Phase 5: Integration & Testing	End-to-end pipeline integration; performance benchmarking; threshold calibration; documentation	1 week

10.2 Data Flow

The system's data flow proceeds as follows. The VideoCapture object continuously reads frames from the video source and places them into a processing queue. A dedicated processing thread dequeues frames, applies frame skipping logic, and passes qualifying frames to the YOLOv8 model for inference. The detection results are filtered, counted, and smoothed by the counting module. The smoothed count is classified by the density module. The resulting occupancy record — comprising count, density label, and timestamp — is written to an in-memory data store (e.g., a thread-safe queue or Redis cache). The FastAPI server reads from this data store and serves the most recent record to polling dashboard clients, or pushes updates via WebSocket to connected clients. The Streamlit dashboard reads the API response and updates its displayed metrics accordingly.

10.3 Model Usage

The system uses pre-trained YOLOv8 weights trained on the COCO dataset, specifically utilising the person class (class index 0 in COCO nomenclature). No custom training or fine-tuning is required for the initial deployment, as the COCO person class encompasses the diversity of human appearances encountered in railway station environments. Should deployment in an environment with unusual camera angles or passenger attire (e.g., heavy winter clothing obscuring body contours) result in unsatisfactory detection performance, fine-tuning on a small custom dataset of platform imagery can be performed using Ultralytics' training API with as few as 500 annotated images.

11. Expected Output

11.1 Real-Time Detection Output

At the frame level, the system produces annotated video frames in which each detected person is enclosed by a green bounding box labelled with the confidence score. The total person count for the frame is overlaid in the upper left corner of the annotated frame. When operating in dashboard mode, these annotated frames may optionally be streamed to the dashboard via MJPEG or WebRTC for visual confirmation of detection performance.

11.2 Crowd Density Classification

The system's primary operational output is a density classification label accompanied by the person count and timestamp. A representative sample of expected output records is illustrated in the table below:

Timestamp	Person Count	Density Classification
09:02:15	7	Low
09:15:42	23	Medium
09:28:07	51	High
09:45:30	18	Medium
10:12:55	4	Low

11.3 Dashboard Features

The Streamlit dashboard presents operations staff with the following information elements. A prominently displayed metric card shows the current person count and density classification, colour-coded to provide immediate visual status awareness. A time-series line chart displays the evolution of occupancy over the preceding 30-minute window, enabling operators to identify trends and anticipate emerging crowd surges. A status indicator banner displays alert messages when High density is detected, triggering visible and optionally audible notifications. Summary statistics including the session maximum, minimum, and average person count are displayed in secondary metric cards. An optional annotated video feed panel shows the live detection output directly in the dashboard interface.

12. Advantages

12.1 Real-Time Operational Awareness

The most significant operational advantage of the proposed system is the continuous, objective, real-time occupancy awareness it provides to station staff. Unlike manual observation, the system operates without fatigue and delivers quantitative metrics rather than subjective impressions, enabling data-driven decision-making. The low latency of the processing pipeline — typically less than 200 milliseconds from frame capture to dashboard update on GPU hardware — ensures that occupancy information is current enough to support time-critical operational responses.

12.2 Automation of Surveillance

By automating the analytical layer of CCTV surveillance, the system releases control room operators from the cognitively demanding task of manually monitoring crowd levels across multiple camera feeds. Instead, operators receive processed, classified data and are alerted only when intervention thresholds are breached. This reduces cognitive load, improves response consistency, and allows staff to focus their attention on other operational responsibilities during periods of low occupancy.

12.3 Safety Enhancement

The system's threshold-based alerting mechanism enables proactive safety interventions before crowd density reaches dangerous levels. Automated alerts can trigger a range of predefined responses, including broadcasting passenger advisory announcements over the public address system, activating dynamic signage to redirect passengers to less congested platforms, deploying additional platform staff, or temporarily restricting platform access. These early interventions have the potential to prevent crowd crush incidents and reduce the risk of passenger falls or altercations in congested areas.

12.4 Data-Driven Capacity Planning

The historical occupancy data generated by the system provides a quantitative basis for long-term capacity planning. Analysing occupancy patterns across days, weeks, and seasons enables transit authorities to optimise train scheduling, staffing allocation, and platform infrastructure investment with a level of empirical grounding that is impossible to achieve through manual observation alone. This data can also inform the parametrisation of simulation models used for station design and emergency evacuation planning.

12.5 Cost-Effectiveness

The system leverages existing CCTV infrastructure, open-source software, and pre-trained models, minimising the capital expenditure required for deployment relative to

proprietary commercial solutions. The modular architecture allows incremental deployment — beginning with a single camera at a high-priority platform — with expansion to additional cameras and stations as budget permits.

13. Limitations

13.1 Occlusion in Dense Crowds

The most fundamental limitation of bounding box-based person detection in dense crowds is occlusion: when passengers stand close together, they partially or fully occlude one another from the camera's perspective, causing the detection model to miss individuals or merge multiple people into a single detection. This limitation is inherent to single-camera, single-perspective detection and tends to result in systematic undercounting of actual occupancy in High density scenarios. Density map estimation approaches are theoretically more robust to this limitation, but at the cost of individual localisation capability.

13.2 Camera Placement Dependency

The accuracy of the detection system is highly sensitive to camera placement. Cameras positioned at overhead angles perpendicular to the platform provide superior coverage, minimising occlusion and presenting passengers as relatively uniform-sized objects in the field of view. Cameras mounted at oblique or near-horizontal angles — as is common with legacy CCTV installations designed for security purposes rather than crowd analysis — introduce significant perspective distortion and object scale variability, reducing detection accuracy. Retrofitting the system to existing non-optimal camera installations may require acceptance of reduced performance or physical repositioning of cameras.

13.3 Lighting and Environmental Conditions

YOLOv8, like all CNN-based detectors, is sensitive to significant changes in scene illumination. Outdoor platforms subject to direct sunlight, shadows, and the transition between day and night conditions present challenges that may reduce detection confidence for individuals partially in shadow. Artificial lighting conditions in underground or covered stations, including the presence of sodium vapour or mercury vapour lamps that produce non-standard spectral compositions, may also affect detection performance if the training data distribution does not represent these conditions adequately.

13.4 Non-Person Object Confusion

In railway environments, large pieces of luggage, pushchairs, and maintenance equipment may occasionally generate false positive person detections if the model's confidence threshold is set too permissively. Conversely, passengers wearing unusually large coats, carrying oversized bags, or grouped in close physical proximity may present as a single detection. These edge cases, while infrequent, contribute to counting inaccuracy and require ongoing threshold calibration.

13.5 Computational Resource Requirements

While the system is designed to operate on standard hardware, real-time processing of high-resolution video remains computationally demanding. On CPU-only hardware, achieving acceptable inference latency may require reducing video resolution or increasing frame skip intervals, both of which reduce detection granularity. Organisations deploying the system on resource-constrained hardware must accept a trade-off between processing throughput and detection accuracy.

14. Future Enhancements

14.1 Spatial Heatmap Generation

A natural extension of the current bounding box-based detection approach is the generation of spatial occupancy heatmaps that visualise the distribution of passengers across the platform surface. By accumulating the centroid coordinates of detected persons over a configurable time window and applying Gaussian kernel density estimation, the system can produce a continuous heat map overlay on the camera's field of view. This provides operations staff with spatial awareness of crowd hotspots — areas of locally elevated density that may not be apparent from aggregate count metrics alone — enabling more targeted crowd management interventions.

14.2 Multi-Camera Integration

The current system architecture is designed for single-camera operation, with the API and dashboard presenting data from a single video feed. A significant operational enhancement would be the extension of the pipeline to ingest and simultaneously process multiple camera feeds, each monitoring a distinct platform zone or a different station altogether. This would require a camera management layer above the existing processing pipeline, with the dashboard presenting a multi-camera overview that summarises occupancy across all monitored zones. A unified station-level occupancy score could be derived from zone-level counts, providing a single metric for overall platform capacity status.

14.3 Predictive Occupancy Modelling

Historical occupancy time-series data generated by the system provides the foundation for predictive modelling. Time-series forecasting algorithms such as LSTM-based recurrent neural networks, Facebook Prophet, or ARIMA models could be trained on the historical data to predict occupancy levels at future time points — for example, 15 or 30 minutes ahead. Predictive occupancy alerts would enable operations staff to implement crowd management measures in anticipation of surges rather than in reaction to them, further enhancing the proactive safety benefits of the system.

14.4 Smart Alert Integration

The current alert mechanism is limited to visual indicators on the dashboard. Future enhancements could integrate the alert system with external communication infrastructure, including automatic generation of public address announcements, transmission of SMS or push notification alerts to mobile devices carried by platform staff, and automatic submission of occupancy data to a central station management system. Integration with digital signage controllers could enable dynamic adjustment of passenger information displays to guide passengers toward less congested areas based on real-time occupancy data.

14.5 Individual Tracking and Dwell Time Analysis

Incorporating a multi-object tracking algorithm — such as DeepSORT or ByteTrack — into the processing pipeline would enable the system to assign persistent identities to detected individuals across consecutive frames. This capability would support dwell time analysis (measuring how long passengers remain on the platform), entry and exit counting (distinguishing between passengers arriving on the platform and departing), and trajectory analysis (identifying common movement patterns and potential bottlenecks). These capabilities would significantly enrich the operational insights available from the system beyond simple instantaneous occupancy counts.

15. Applications

15.1 Railway Stations

The primary application domain for the proposed system is railway station platform monitoring, as described throughout this document. The system is applicable to a broad range of railway contexts, from urban metro and rapid transit stations with very high passenger throughput to intercity mainline platforms where surge events are concentrated around train arrivals and departures. The configurable density classification thresholds allow the system to be calibrated for the specific capacity characteristics of each platform.

15.2 Airports and Transit Hubs

The system's fundamental capability — real-time detection and counting of people in a defined area — is directly applicable to airport terminal monitoring, specifically for queue management at check-in counters, security screening lanes, and boarding gates. In larger transit hubs combining rail, bus, and ferry services, the system could provide a unified occupancy awareness layer across multiple transport modalities.

15.3 Public Events and Venues

Large public gatherings such as concerts, sporting events, and political rallies present crowd management challenges analogous to those encountered in railway stations. The system could be deployed at entry and exit points of event venues to monitor queue buildup and provide real-time occupancy data to event safety officers, supporting compliance with venue capacity regulations and enabling proactive management of crowd flow.

15.4 Smart Cities and Urban Infrastructure

At a broader urban scale, the system represents a component of the smart city vision in which public spaces are continuously monitored and managed through automated sensor networks and AI-powered analytics. Deployment across a network of CCTV cameras in pedestrian zones, public squares, and shopping precincts would enable city operators to monitor crowd dynamics in real time, informing dynamic traffic management, emergency service pre-positioning, and public health compliance monitoring.

15.5 Healthcare and Emergency Contexts

Occupancy monitoring based on computer vision has demonstrated utility in healthcare settings, including monitoring waiting room occupancy in hospitals and emergency departments. In emergency contexts, such as post-disaster evacuation centres or emergency shelters, the system could provide real-time capacity information to

emergency management coordinators, facilitating efficient distribution of displaced persons across available facilities.

16. Conclusion

This report has presented a comprehensive design for a Real-Time Platform Occupancy Detection System for Railway Stations, leveraging the state-of-the-art YOLOv8 object detection architecture within a modular, API-first software pipeline. The system addresses a clearly identified operational gap in railway station management: the absence of automated, quantitative, real-time crowd monitoring that can support proactive safety interventions and data-driven capacity planning.

The proposed architecture — encompassing video ingestion via OpenCV, deep learning-based person detection via YOLOv8, density classification, RESTful data exposure via FastAPI, and live dashboard visualisation via Streamlit — is both technically sound and practically accessible. By building upon pre-trained models and open-source frameworks, the system can be deployed at a fraction of the cost of proprietary commercial alternatives, while remaining extensible to accommodate future enhancements such as multi-camera integration, predictive modelling, and spatial heatmap generation.

The limitations of the current design — primarily relating to occlusion in very high-density scenarios, camera placement dependency, and lighting sensitivity — are acknowledged and represent opportunities for future research and engineering effort. The dense crowd environment of a busy railway platform constitutes one of the most challenging scenarios for video-based person detection, and the performance of the system under these conditions should be rigorously validated before operational deployment.

Nevertheless, the potential impact of the system is substantial. By transforming passive CCTV infrastructure into an active, intelligent occupancy monitoring asset, the system has the potential to prevent crowd crush incidents, improve passenger experience during peak periods, and provide transit authorities with the quantitative data necessary to optimise operations and infrastructure investment. The broader applicability of the system architecture to airports, event venues, and urban public spaces further underscores its significance as a contribution to the intelligent transportation systems and smart city domains.

In conclusion, the Real-Time Platform Occupancy Detection System represents a technically rigorous, operationally relevant, and practically deployable solution to a real-world safety and efficiency challenge facing railway operators worldwide. Its

development and deployment would constitute a meaningful advancement in the application of computer vision technology to the management of public transit infrastructure.

Appendix A: Workflow Diagram Explanation

The following text-based workflow diagram illustrates the end-to-end data flow through the system pipeline. Each stage is described in terms of its input, transformation, and output.

SYSTEM WORKFLOW — STAGE-BY-STAGE

STAGE 1: VIDEO SOURCE

Input: RTSP stream / USB camera / video file

OpenCV VideoCapture initialises connection to video source. Frames are continuously read at the native frame rate of the source. Frame dimensions and FPS metadata are logged on initialisation.

Output: Raw BGR image frames (numpy arrays)

STAGE 2: FRAME EXTRACTOR

Input: Raw frame stream from Stage 1

Frame counter increments on each received frame. Every Nth frame (configurable) is selected for processing; remainder are discarded. Selected frame is resized to 640×640 pixels (YOLOv8 input resolution).

Output: Pre-processed frame (640×640 BGR numpy array)

STAGE 3: YOLOv8 DETECTOR

Input: Pre-processed frame from Stage 2

YOLOv8 model.predict() call runs the frame through the backbone, FPN, and detection head. Raw predictions are post-processed with NMS. Result object contains bounding boxes, class indices, and confidence scores for all detections.

Output: Detection result object (boxes, classes, confidences)

STAGE 4: PERSON FILTER & COUNTER

Input: Detection results from Stage 3

Filter detections to class index 0 (person) with confidence >= threshold. Count filtered detections. Apply exponential moving average smoothing to count. Optionally apply ROI mask to restrict counting zone.

Output: Smoothed integer person count, bounding box list

STAGE 5: DENSITY CLASSIFIER

Input: Person count from Stage 4

Compare count against configured thresholds (Low: <15, Medium: 15–40, High: >40). Assign density label. Package count, label, and UTC timestamp into occupancy record.

Output: Occupancy record {count, density, timestamp}

STAGE 6: API SERVICE LAYER

Input: Occupancy record from Stage 5

FastAPI service stores latest occupancy record in memory. GET /occupancy endpoint returns JSON record to polling clients. Optional WebSocket channel pushes updates proactively.

Output: JSON API response {count, density, timestamp}

STAGE 7: STREAMLIT DASHBOARD

Input: JSON response from Stage 6 API

Dashboard polls API at 1–2 second interval (or receives WebSocket push). Updates metric cards, time-series chart, and alert banner. Colour codes density status (Green/Amber/Red).

Output: Live browser-rendered occupancy dashboard

Appendix B: Pseudo-Code — Crowd Counting Logic

The following structured pseudo-code describes the core algorithmic logic of the detection, counting, and classification pipeline. The pseudo-code is written in a language-agnostic style but maps directly to the Python implementation.

CONSTANTS AND CONFIGURATION

```
CONFIDENCE_THRESHOLD   ← 0.40
NMS_IOU_THRESHOLD      ← 0.45
FRAME_SKIP_INTERVAL    ← 3
EMA_ALPHA              ← 0.30
LOW_THRESHOLD          ← 15
HIGH_THRESHOLD         ← 40
PERSON_CLASS_INDEX     ← 0
```

INITIALISATION

```
PROCEDURE Initialise():
    model         ← LoadYOLOv8('yolov8n.pt')
    video_source  ← OpenVideoCapture(SOURCE_URL)
    frame_counter ← 0
    smoothed_count ← 0.0
    history        ← EmptyList()
END PROCEDURE
```

MAIN PROCESSING LOOP

```
PROCEDURE MainLoop():
    WHILE video_source.isOpen() DO
        frame ← video_source.ReadNextFrame()
        IF frame IS NULL THEN BREAK
        frame_counter ← frame_counter + 1
        IF frame_counter MOD FRAME_SKIP_INTERVAL ≠ 0 THEN
CONTINUE

        // ── Detection ──────────────────────────
        resized_frame   ← Resize(frame, 640, 640)
        raw_detections  ← model.Predict(resized_frame)
        filtered        ← FilterDetections(raw_detections)
```

```
        // ── Counting ──────────────────────────────
        raw_count      ← Length(filtered)
        smoothed_count ← ApplyEMA(smoothed_count, raw_count)
        integer_count  ← Round(smoothed_count)

        // ── Classification ────────────────────────
        density_label  ← ClassifyDensity(integer_count)

        // ── Emit Result ───────────────────────────
        record ← {count: integer_count, density: density_label,
timestamp: Now()}
        history.Append(record)
        UpdateAPIDataStore(record)

        // ── Annotate Frame (optional) ─────────────
        annotated ← DrawBoundingBoxes(frame, filtered)
        DrawLabel(annotated, 'Count: ' + integer_count)
    END WHILE
END PROCEDURE
```

HELPER FUNCTIONS

```
FUNCTION FilterDetections(detections):
    result ← EmptyList()
    FOR EACH detection IN detections DO
        IF detection.class_index = PERSON_CLASS_INDEX
           AND detection.confidence >= CONFIDENCE_THRESHOLD
THEN
            result.Append(detection)
    END FOR
    RETURN result
END FUNCTION

FUNCTION ApplyEMA(previous_smooth, new_raw):
    RETURN EMA_ALPHA * new_raw + (1 - EMA_ALPHA) *
previous_smooth
END FUNCTION
```

```
FUNCTION ClassifyDensity(count):
    IF count < LOW_THRESHOLD THEN
        RETURN 'Low'
    ELSE IF count <= HIGH_THRESHOLD THEN
        RETURN 'Medium'
    ELSE
        RETURN 'High'
    END IF
END FUNCTION
```

References

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.

Zhang, Y., Zhou, D., Chen, S., Gao, S., & Ma, Y. (2016). Single-Image Crowd Counting via Multi-Column Convolutional Neural Network. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 589–597.

Li, Y., Zhang, X., & Chen, D. (2018). CSRNet: Dilated Convolutional Neural Networks for Understanding the Highly Congested Scenes. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1091–1100.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. GitHub. https://github.com/ultralytics/ultralytics

Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools, 25(11), 120–125.

Cao, X., Wang, Z., Zhao, Y., & Su, F. (2018). Scale Aggregation Network for Accurate and Efficient Crowd Counting. Proceedings of the European Conference on Computer Vision (ECCV), 734–750.

Wojke, N., Bewley, A., & Paulus, D. (2017). Simple Online and Realtime Tracking with a Deep Association Metric. Proceedings of the IEEE International Conference on Image Processing (ICIP), 3645–3649.

Singh, A., Kumar, R., & Verma, P. (2022). Real-Time Crowd Monitoring in Railway Stations Using YOLOv5. International Journal of Computer Vision and Image Processing, 12(3), 45–62.

Patel, S., & Shah, M. (2021). Automated Platform Occupancy Detection Using Faster R-CNN for Railway Safety Management. Proceedings of the International Conference on Intelligent Transportation Systems (ITSC), 1234–1241.